PROGRAMAÇÃO WEB

Sistema de cadastro

CRUD com autenticação de login em PHP e MySQL

Projeto de Gabi

Apresentação técnica para a banca de professores

Ambiente de Desenvolvimento / Servidor local

XAMPP: Apache + MySQL

Apache

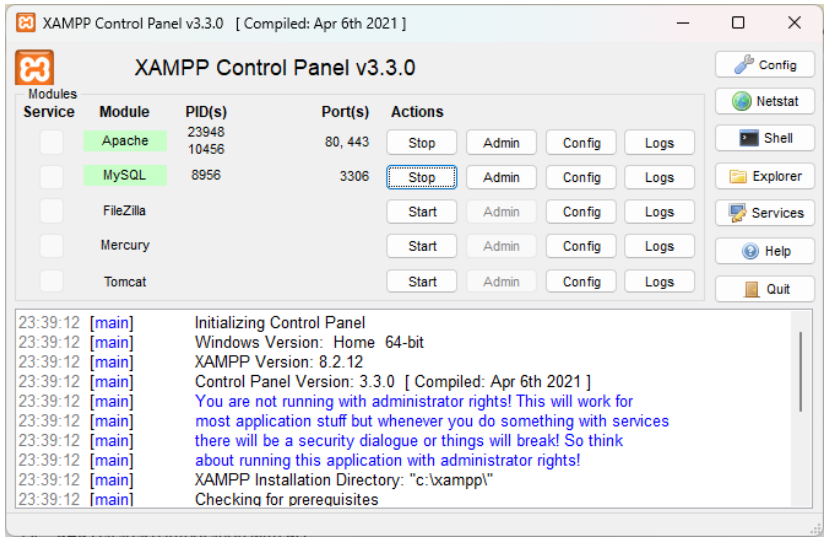
Servidor web ativo na porta 80/443

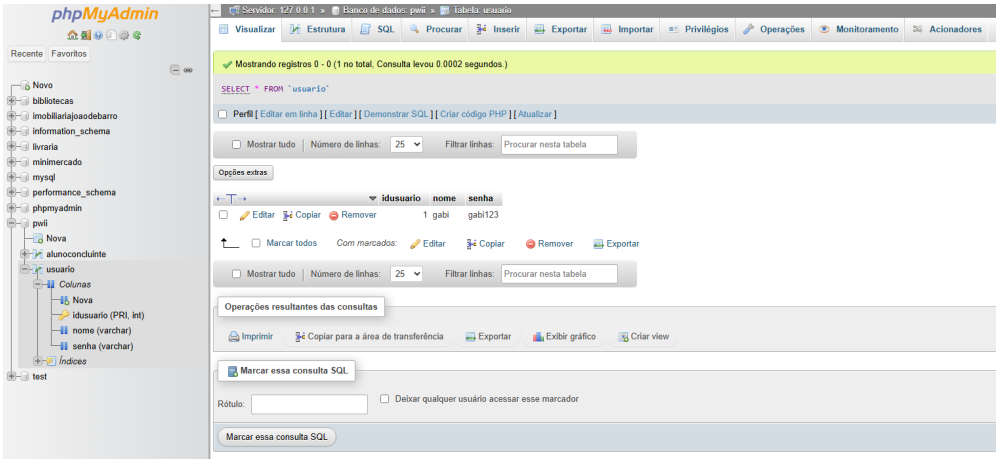
MySQL

Banco de dados ativo na porta 3306

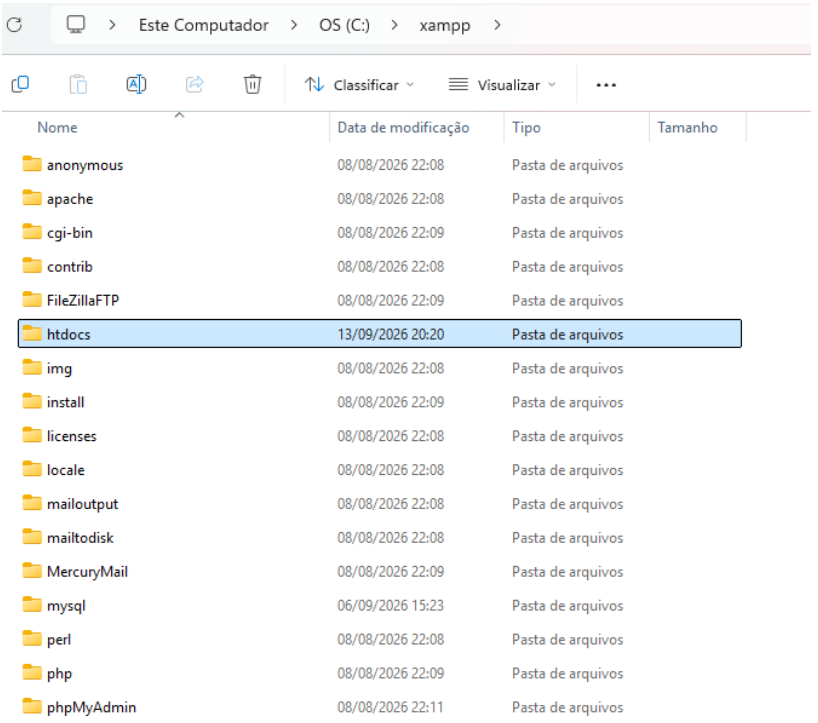
phpMyAdmin

Interface para gerenciar o banco pwii





Estrutura de pastas em C:\xampp\htdocs



É dentro de htdocs que ficam todos os arquivos PHP do projeto.

Modelagem da tabela usuario

```sql
CREATE TABLE usuario (
    idusuario INT(11) NOT NULL AUTO_INCREMENT,
    nome VARCHAR(45) NOT NULL,
    senha VARCHAR(45) NOT NULL,
    PRIMARY KEY (idusuario)
);
```

idusuario

Chave primária, gerada automaticamente

nome

Login do usuário (até 45 caracteres)

senha

Senha do usuário (até 45 caracteres)

```sql
INSERT INTO pwii.usuario (nome, senha) VALUES ('gabi', 'gabi123');
```

Populando a tabela

Dados teste de Gabi

Código - Fluxo

index.php

Usuário digita nome e senha

→

loginAction.php

Valida os dados no banco MySQL

→

principal.php

Acesso liberado com verificarAcesso.php

→

logoutAction.php

Encerra a sessão e retorna ao login

- **Formulário simples**

  Campos de usuário e senha, mais um botão de envio

- **Framework W3.CSS**

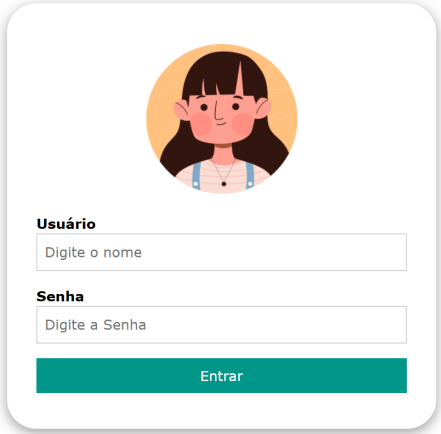
  Utilizado para estilização rápida e responsiva

- **method="post"**

  Os dados são enviados de forma segura no corpo da requisição

- **action="loginAction.php"**

  Aponta para o script que processa a autenticação



gabi, seja bem-vinda!

Testando o código

O código PHP do projeto foi todo escrito no Visual Studio Code. Para testar, com o Apache e o MySQL ativos no XAMPP, acesse http://localhost/nome-da-pasta/ no navegador (certificando que está salvo na pasta (C:\xampp\htdocs).

Protegendo as páginas internas

verificarAcesso.php

Para impedir que alguém acesse páginas internas sem passar pelo login, esse trecho é repetido no início de cada arquivo protegido.

```php
if (!isset($_SESSION['logado'])
    == true) {
  header('location: acessoNegado.php');
  die();
}
```

acessoNegado.php

```php
<?php require_once ('cabecalho.php'); ?>
<div class="w3-padding w3-content w3-text-grey w3-third w3-display-middle">
    <?php
    echo '
    <a href="index.php" style="text-decoration: none;">
        <h1 class="w3-button w3-teal">Acesso NEGADO!</h1>
        <br>
        <h2 class="w3-button w3-teal">É necessária a Realização do Login!</h2>
    </a>
    ';
    ?>
</div>
<?php require_once ('rodape.php'); ?>
```

Se não encontrar a confirmação de login, bloqueia o acesso imediatamente e envia a pessoa para a tela de acesso negado.



Back-end

Autenticação de login

1

Recebe os dados

Os campos txtNome e txtSenha, vistos no formulário da tela de login, são enviados via method="post" para loginAction.php. Lá, o PHP acessa esses valores através da variável superglobal $_POST, guardando-os em $nome e $senha para uso na validação.

2

Conecta ao banco

O loginAction.php inclui conexaoBD.php, que centraliza os dados de acesso ao banco (servidor, usuário, senha) e é reutilizado sempre que alguma página do sistema precisa se conectar ao MySQL

3

Consulta o usuário

O sistema procura na tabela usuario um registro com o nome digitado, usando o comando SELECT. Se encontrar, retorna os dados desse usuário (incluindo a senha) para a próxima etapa de verificação.

4

Compara a senha

O PHP compara a senha digitada com a senha armazenada no banco. Se forem iguais, o acesso é liberado para principal.php; caso contrário, a página exibe "Login inválido!".

Sessão ativa

Enquanto $_SESSION['logado'] estiver ativa, o usuário navega livremente pelas páginas protegidas do CRUD de amigos.

Encerrando a sessão

logoutAction.php remove a variável de sessão com unset($_SESSION['logado']) e redireciona de volta ao index.php.